SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5102
NAME:PREETHI R
REG NO:192511352

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I _Preethi R(192511352)___(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

---

2. Digital Signature for Document Authentication
A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

---

3. Preventing Replay Attacks in Online Transactions
An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.
B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

---

4. Secure Communication over Public Wi-Fi
A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.
B. Recommend a secure communication method and justify how it protects user data.

---

5. Data Integrity in Cloud Storage
An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.
B. Suggest an additional mechanism to enhance trust and justify your answer.

1. Secure Password Storage in Web Applications

A. Explain why storing plaintext passwords is insecure.

Plaintext password storage means storing users' passwords in their original, readable form in a database without applying any security mechanism.

Storing passwords in plaintext is insecure because if an attacker gains access to the database, all passwords can be viewed directly. This may lead to unauthorized access to user accounts, theft of personal information, financial loss, and privacy violations.

The major reasons are:

1. Easy access for attackers: If the database is compromised, attackers can immediately read all stored passwords without needing to crack them.
2. Unauthorized account access: Attackers can use exposed passwords to log in to users' accounts and misuse their personal information.
3. Password reuse: Users often use the same password for multiple websites. A password stolen from one application may be used to access other accounts.
4. Privacy risks: Personal details, confidential information, and financial data may be exposed when attackers access user accounts.
5. Insider threats: Employees or administrators with database access may view users' passwords if they are stored in plaintext.
6. Loss of trust: A password leak can damage the reputation of an organization and may lead to legal or financial consequences.

For example, if a website stores a user's password as Welcome123, an attacker who accesses the database can read the password directly and use it to log in.

Therefore, plaintext password storage should be avoided. Passwords must be protected using secure password-hashing techniques.

B. Suggest a secure method for storing passwords and justify your choice.

A secure method for storing passwords is salted password hashing using Argon2id.

Password hashing converts a password into a fixed-format hash value using a one-way cryptographic function. During login, the application verifies the entered password against the stored hash instead of retrieving the original password.

A salt is a unique, randomly generated value used along with each password during hashing. It ensures that identical passwords can produce different hash values when different salts are used.

The process is as follows:

1. The user creates an account and enters a password.
2. The application generates a unique random salt.
3. The password and salt are processed using the Argon2id password-hashing algorithm.
4. The resulting hash and necessary parameters are stored in the database instead of the original password.
5. During login, the application verifies the entered password using the stored information.
6. If verification succeeds, the user is allowed to log in; otherwise, access is denied.

Justification:

- Protection against database breaches: Attackers cannot immediately read the original passwords from the stored hashes.
- One-way process: Password hashing is designed to make recovering the original password computationally difficult.
- Unique salt: It prevents identical passwords from having identical stored hashes when different salts are used.
- Resistance to password cracking: Argon2id uses memory and computational resources to make large-scale password guessing more expensive.
- Secure verification: The application can verify passwords without storing them in readable form.

For additional security, the application should use HTTPS, enforce login-attempt limits, restrict database access, and enable multi-factor authentication where appropriate.

Thus, salted Argon2id password hashing is a suitable method because it reduces the risks associated with stolen password databases and protects users' credentials more effectively than plaintext storage.

2. Digital Signature for Document Authentication

A. Explain how digital signatures can be used in this scenario.

A digital signature is a cryptographic technique used to verify the authenticity and integrity of electronic documents. It uses public-key cryptography, consisting of a private key for signing and a corresponding public key for verification.

A company can use digital signatures to sign official documents such as contracts, invoices, certificates, and financial reports before sending them electronically.

The process works as follows:

1. Key generation: The sender obtains a public-private key pair. The private key is kept secure and used to create the signature, while the public key is used to verify it.
2. Document hashing: The sender applies a cryptographic hash function, such as SHA-256, to the document to generate a message digest.
3. Signature generation: The sender uses the private key and a digital-signature algorithm to generate a digital signature associated with the document.
4. Document transmission: The sender sends the document along with its digital signature to the recipient through email or another electronic communication system.
5. Signature verification: The recipient uses the sender's public key and the appropriate verification algorithm to check whether the signature is valid for the received document.
6. Result checking: If verification succeeds and the sender's identity is trusted, the recipient can accept the document as authentic and unaltered. If verification fails, the document may have been modified or the signature may be invalid.

Example: Suppose a company sends an electronically signed employment contract to an employee. The company signs the contract using its authorized private key. When the employee receives the document, the signature is verified using the corresponding public key.

If someone modifies the salary or employment terms after signing, the signature verification will normally fail. This helps the employee identify unauthorized changes and verify the authenticity of the signed document.

Therefore, digital signatures provide a reliable way to authenticate official documents exchanged electronically.

B. Justify how this method ensures both integrity and non-repudiation.

Digital signatures provide two important security properties: integrity and non-repudiation.

1. Integrity

Integrity ensures that a document has not been altered or modified without authorization.

A digital signature supports integrity by associating the signature with the document's contents through a cryptographic hash and signature algorithm. When the recipient verifies the signature, any modification to the signed content will normally cause verification to fail.

For example, if an invoice originally contains an amount of ₹50,000 and an attacker changes it to ₹80,000 after it has been signed, the altered document will not verify against the original signature.

Thus, digital signatures help detect unauthorized modifications and ensure that the received document matches the signed content.

2. Non-repudiation

Non-repudiation means providing evidence that helps prevent a signer from denying that they signed a particular document.

A digital signature supports non-repudiation because it is generated using the signer's private key and can be verified using the corresponding public key.

For example, if a company signs a purchase agreement digitally and later a dispute occurs, the recipient can present the signed document and verify the signature using the company's public key. A trusted digital certificate and secure audit records can provide additional evidence connecting the signature to the company.

Thus, digital signatures help establish accountability and provide evidence of signing.

However, non-repudiation depends on protecting the private key, reliably identifying the signer, and maintaining appropriate records. A valid signature alone cannot guarantee which person used a compromised or shared private key.

Conclusion

Digital signatures ensure integrity by helping detect unauthorized changes to signed documents. They support non-repudiation by providing verifiable evidence associated with the signer's private key. Therefore, they are useful for authenticating official documents, protecting electronic business transactions, and establishing trust between organizations.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand